Smart Patio Shield — Deployment Artifact: Setup & Walkthrough

This is your live inference demo for the presentation. It has two parts:

1. `app.py` — a small backend that loads your **real trained XGBoost model** and serves predictions.
2. `dashboard.html` — an interactive console the audience can manipulate: drag sliders for weather conditions, or pick a real historical hour, and watch the deploy-or-stow decision update live.

There's a built-in **safety net**: if the backend isn't running, the dashboard automatically switches to an offline demonstration mode using a faithful surrogate of the model's logic. So the demo *never dies* mid-presentation.

Why this design (say this if asked)

The project's central finding was that **satellite imagery adds no predictive value beyond the free reanalysis weather data** (Sections 8–9 of the technical documentation). So the deployed service runs **Model 1 (tabular XGBoost) alone** — a deliberate engineering decision that follows directly from the research result and keeps the live system lightweight: no satellite pipeline, no GPU, just weather inputs in and a decision out. *You deployed what your research showed was worth deploying.* That is a strong, defensible narrative.

One-time setup (about 5 minutes)

You need Python (you already have it) and a few packages.

bash

```
# 1. Put these three files in one folder, together with your trained model:
#      app.py
#      dashboard.html
#      xgboost_baseline.json          <- copy from your models/ folder
#      baseline_training.manifest.json <- copy from models/ (optional but recom

# 2. Install the backend dependencies (once):
pip install fastapi uvicorn xgboost pandas numpy

# 3. (Optional) generate real historical hours for the picker:
#    run this where your processed data lives, then copy sample_hours.json
#    into the demo folder. If you skip it, the picker uses built-in realistic h
python make_sample_hours.py
```

Running the demo

Two steps, every time:

bash

```
# Step 1 — start the model backend (leave this terminal running)
python app.py
#   You should see: "Model loaded. Expecting N columns."
#   and: "Uvicorn running on http://127.0.0.1:8000"

# Step 2 — open the dashboard
#   Just double-click dashboard.html, OR open it in your browser.
```

When the dashboard loads, the status dot (top right) should turn teal and say **"live model connected."** That means it's using your real model. If it says "offline demo mode" in amber, the backend isn't reachable — check that `python app.py` is still running in the other terminal.

> **Tip for the presentation:** open the dashboard *after* the backend is running, and confirm the dot is teal before you begin. If anything goes wrong live, the offline mode kicks in automatically and looks identical — nobody will know.

What to show the markers (a 3-minute script)

1. Start dry, then make it rain. Open on Montego Bay, afternoon, moderate cloud — it should read **STOW**. Now drag **"Rain last hour"** up. Watch the probability climb and the panel flip to a glowing **DEPLOY**. Point out the driver chips updating underneath — *"the*

model is telling you why: recent rain is the dominant signal, exactly as the feature-importance analysis found."

2. Show the operating point. Drag the **decision threshold** slider. Explain: *"This is the deploy/stow cut-off. Lower it and the system becomes more cautious — it deploys on weaker signals but raises more false alarms. This is the precision/recall trade-off from the documentation, exposed as a business decision."*

3. Show location matters. Switch from Montego Bay to Kingston with the same conditions — the probability drops. *"The model learned the rain-shadow effect: the drier south coast has a lower base rate."*

4. Prove it on real data. Switch to **"Real historical hour"** and pick the **Hurricane Melissa** hour. The model says DEPLOY, and the green ✓ confirms the veranda actually got wet. Then pick a **calm dry night** — STOW, and ✓ it actually stayed dry. *"These are genuine test-set hours the model never trained on."*

How it works (for technical questions)

- **The dashboard exposes ~11 intuitive controls** (location, hour, cloud, humidity, recent rain, gusts, pressure trend). The backend expands these into the **full 30-plus feature vector** the model expects — deriving the rest (dew point, VPD, cyclical time encodings, lag terms) with transparent physical formulas. This is why you manipulate *human-understandable* quantities, not `hour_sin`.
- **The prediction is your real XGBoost booster** (`predict_proba`), returning the wind-driven-rain probability. The deploy/stow decision is that probability against the threshold.
- **The offline surrogate** (used only if the backend is down) is a logistic approximation built from the documented feature importances — precip persistence dominant, then cloud, gusts, diurnal cycle, pressure, and the location base rate. It's labelled as a demonstration stand-in, not the real model. It exists purely so a network or process hiccup never breaks your live demo.

Honest limitations (have these ready)

- The demo derives unexposed features from the exposed ones, so it's a **faithful interface to the model, not a live weather feed**. A production system would pull real-time Open-Meteo data and compute the true lag features from the preceding hours.
- The decision threshold is set at 0.5 by default; the right operating point depends on the relative cost of a missed wetting versus an unnecessary deployment, which is a deployment-tuning question, not a modelling one.
- This runs Model 1 only, by design (see "Why this design" above).

File checklist for submission

```
deploy/
  app.py                        # the inference service (real model)
  dashboard.html                # the interactive console
  make_sample_hours.py          # helper to generate real demo hours
  xgboost_baseline.json         # your trained model  (copy in)
  baseline_training.manifest.json  # column order    (copy in)
  sample_hours.json             # generated demo hours (optional)
  README_DEPLOYMENT.md          # this file
```